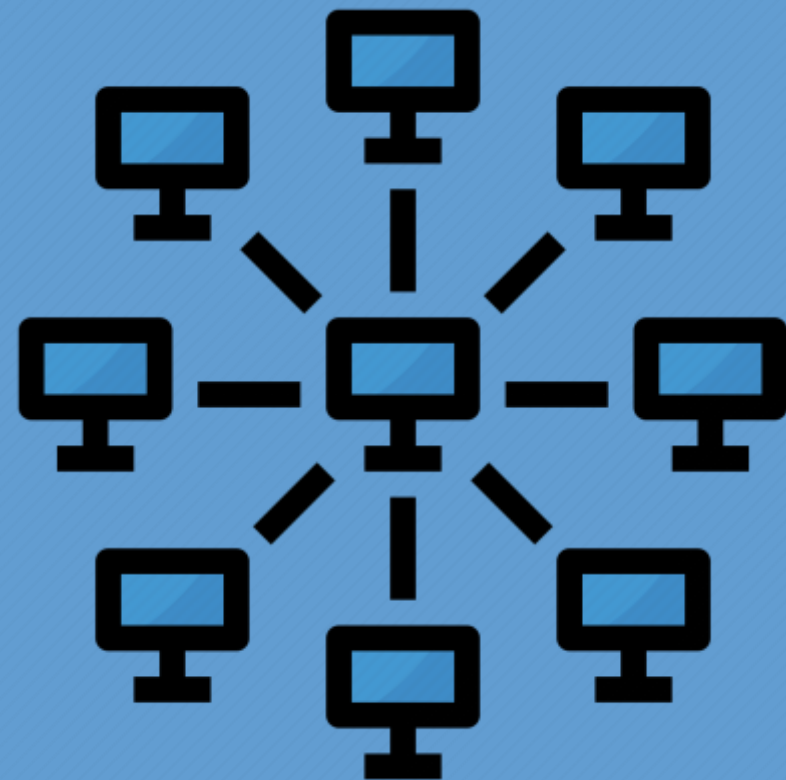


ОСНОВЫ СЕТЕВЫХ ТЕХНОЛОГИЙ. ЧАСТЬ 1



ЛЕКЦИЯ 10. ТРАНСПОРТНЫЙ УРОВЕНЬ

КАФЕДРА
ТЕЛЕКОММУНИКАЦИЙ

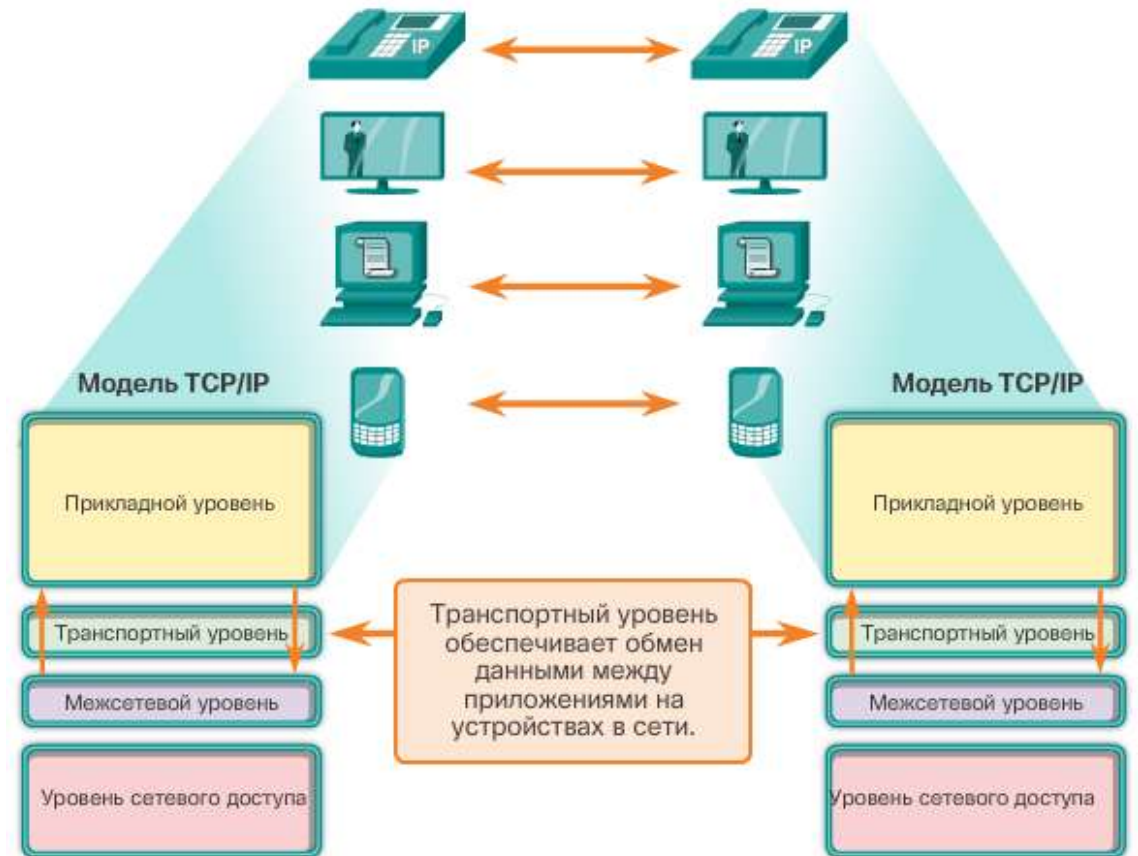
10.1. ПЕРЕДАЧА ДАННЫХ

10.1.1. РОЛЬ ТРАНСПОРТНОГО УРОВНЯ

Транспортный уровень отвечает за установление временного сеанса связи и передачу данных между двумя приложениями.

Транспортный уровень обеспечивает следующее:

1. Поддержка потока данных с установлением соединения
2. Надежность
3. Управление потоком
4. Мультиплексирование

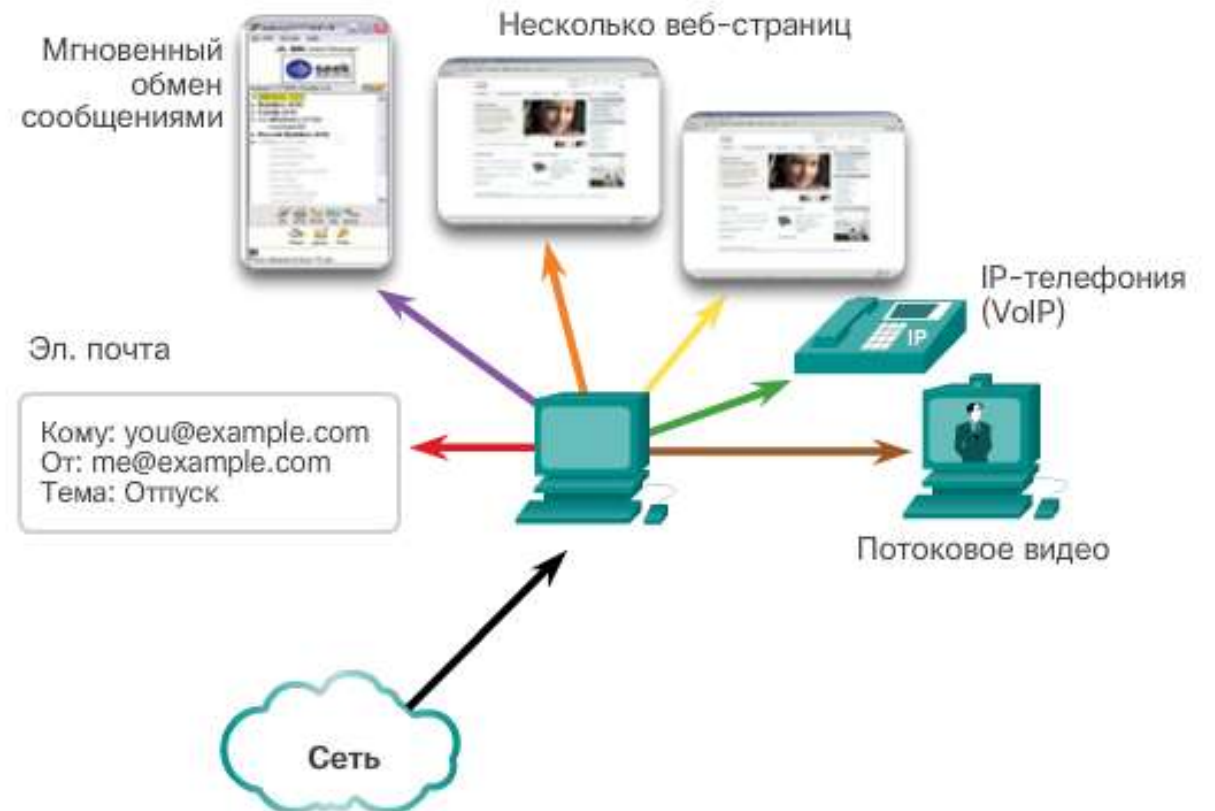


10.1. ПЕРЕДАЧА ДАННЫХ

10.1.2. ФУНКЦИИ ТРАНСПОРТНОГО УРОВНЯ

Отслеживание отдельных сеансов связи

Транспортный уровень отдельно отслеживает каждый индивидуальный сеанс связи между приложением источника и приложением назначения.



10.1. ПЕРЕДАЧА ДАННЫХ

10.1.2. ФУНКЦИИ ТРАНСПОРТНОГО УРОВНЯ

Сегментация данных и последующая сборка сегментов

Транспортный уровень разделяет данные на сегменты, которые проще контролировать и передавать.



10.1. ПЕРЕДАЧА ДАННЫХ

10.1.2. ФУНКЦИИ ТРАНСПОРТНОГО УРОВНЯ

Определение приложений

Транспортный уровень гарантирует, что даже в случае запуска на устройстве нескольких приложений все они получают правильные данные.

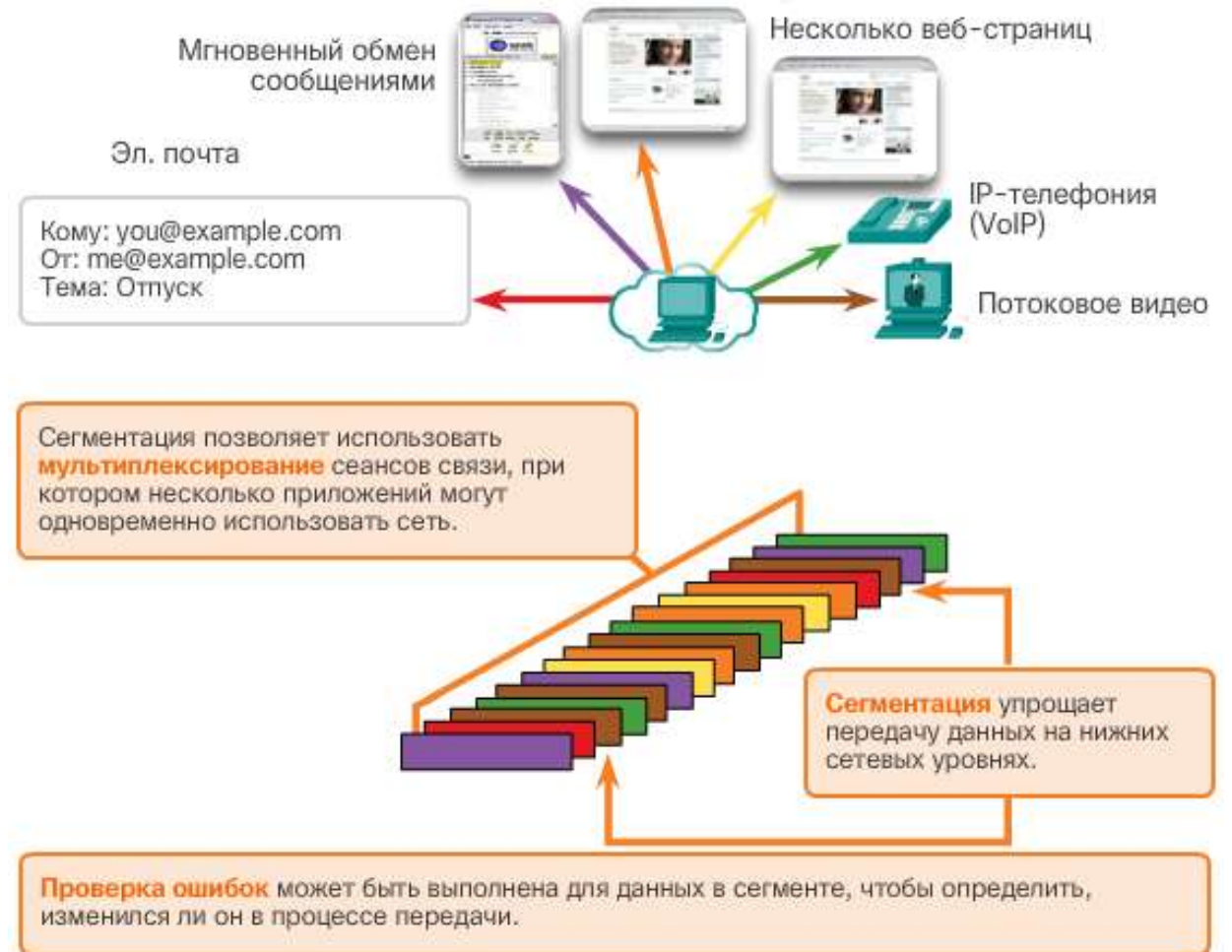


10.1. ПЕРЕДАЧА ДАННЫХ

10.1.3. МУЛЬТИПЛЕКСИРОВАНИЕ СЕАНСОВ СВЯЗИ

Сегментация данных на более мелкие блоки позволяет осуществлять чередовать (мультиплексировать) большое количество разных процессов передачи данных от разных пользователей в одной и той же сети.

Транспортный уровень добавляет заголовок, содержащий двоичные данные для идентификации каждого сегмента данных, и позволяет разным протоколам транспортного уровня выполнять разные функции при управлении передачей данных.



10.1. ПЕРЕДАЧА ДАННЫХ

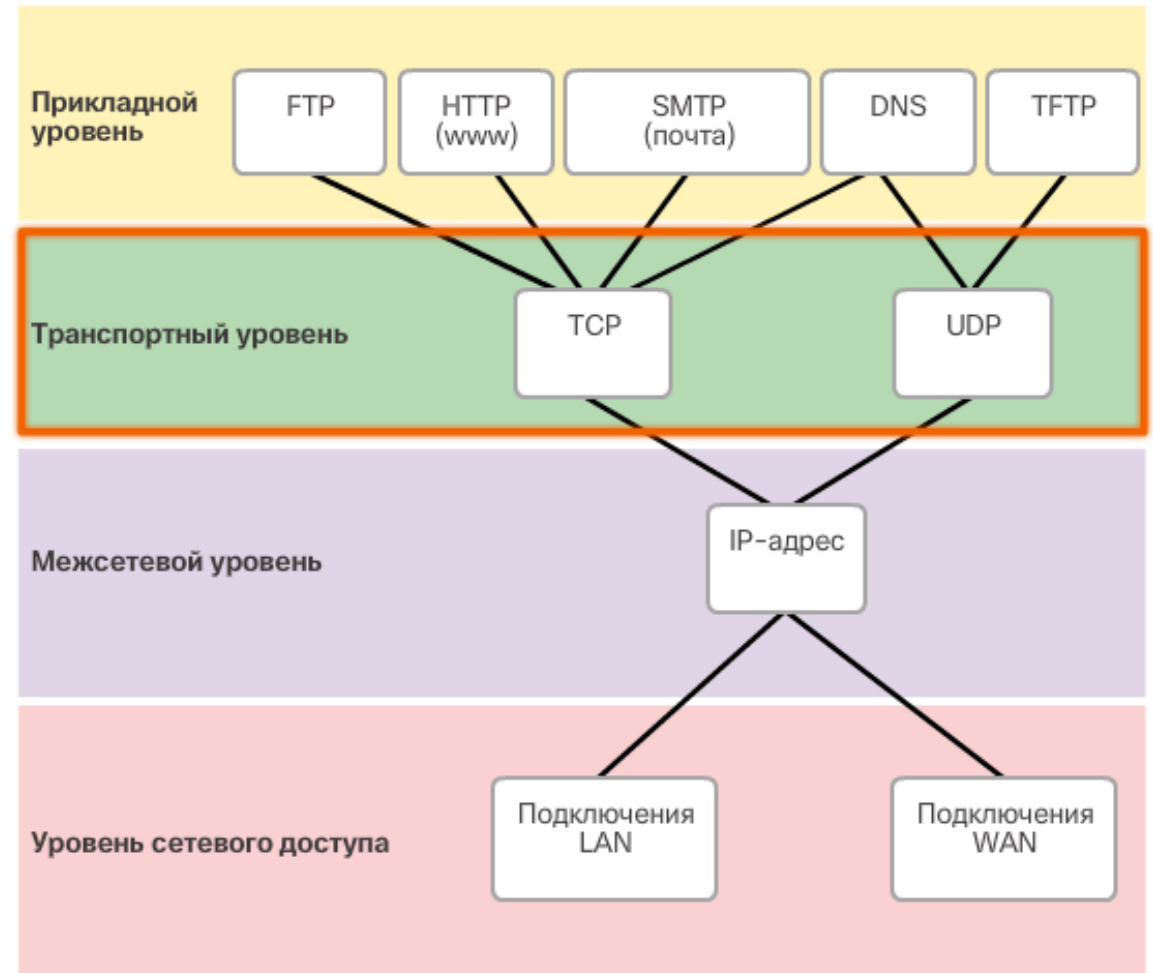
10.1.4. НАДЕЖНОСТЬ ТРАНСПОРТНОГО УРОВНЯ

Транспортный уровень также отвечает за обеспечение **надежности сеанса связи**.

Некоторые приложения могут не предъявлять особых требований к надежности.

Требования к надежности передачи данных зависят от конкретного приложения.

Как показано на рисунке, TCP/IP предоставляет два протокола транспортного уровня: **TCP** (протокол управления передачей) и **UDP** (протокол передачи датаграмм пользователя).



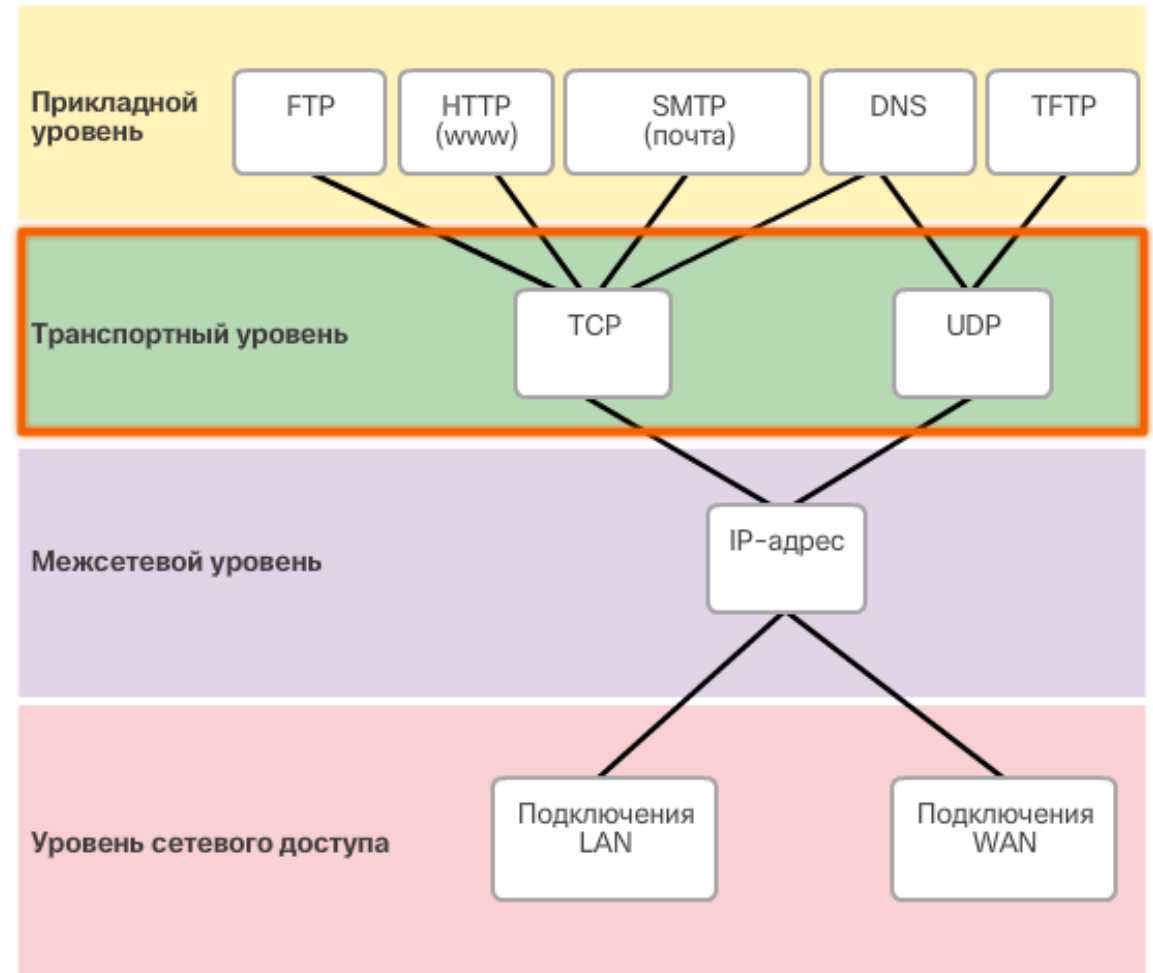
10.1. ПЕРЕДАЧА ДАННЫХ

10.1.4. НАДЕЖНОСТЬ ТРАНСПОРТНОГО УРОВНЯ

Протокол IP использует эти транспортные протоколы для обеспечения связи и передачи данных между узлами.

TCP считается надежным полнофункциональным протоколом транспортного уровня, который позволяет подтверждать доставку пакета данных

UDP в отличие от него – очень простой протокол транспортного уровня, не гарантирующий надежность



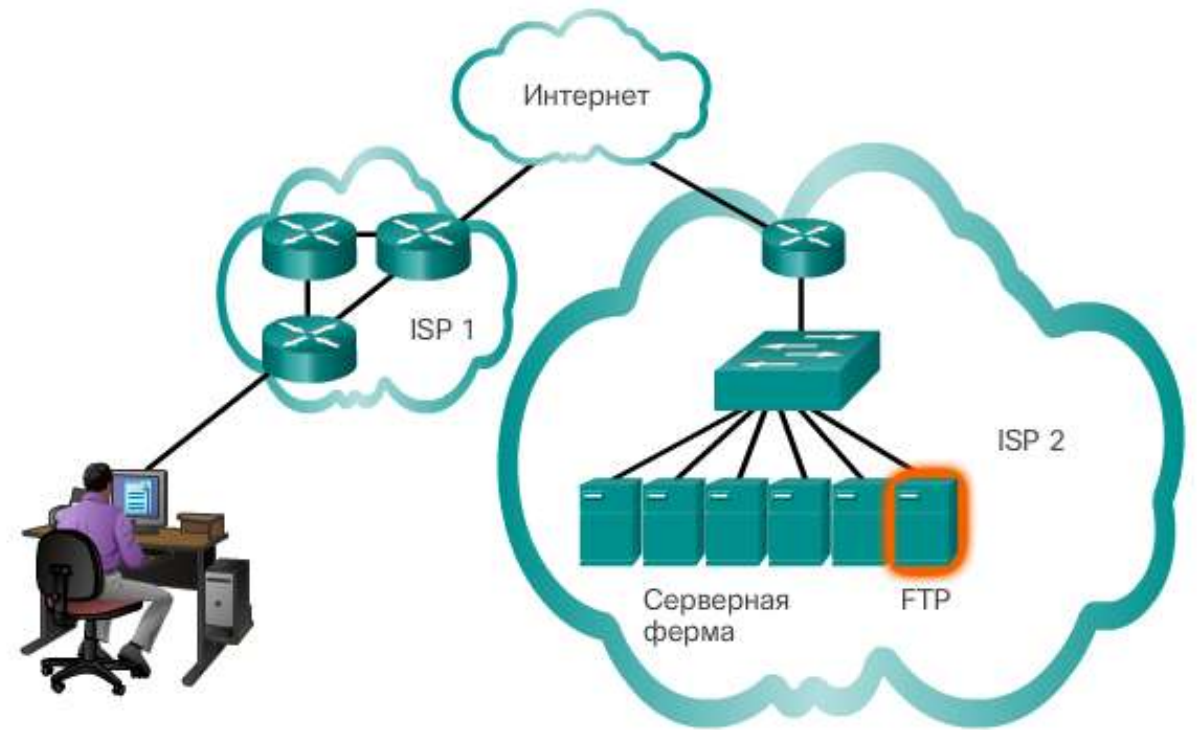
10.1. ПЕРЕДАЧА ДАННЫХ

10.1.5. ПРОТОКОЛ ТСР

Передача данных по протоколу **ТСР** **надежна**, поскольку этот протокол поддерживает подтверждение доставки пакетов.

Существуют три основные операции, обеспечивающие надежность протокола ТСР:

1. Отслеживание количества сегментов, отправленных на тот или иной узел тем или иным приложением
2. Подтверждение получения данных
3. Повторная передача сегментов с неподтвержденными данными по истечении определенного времени (таймаута)

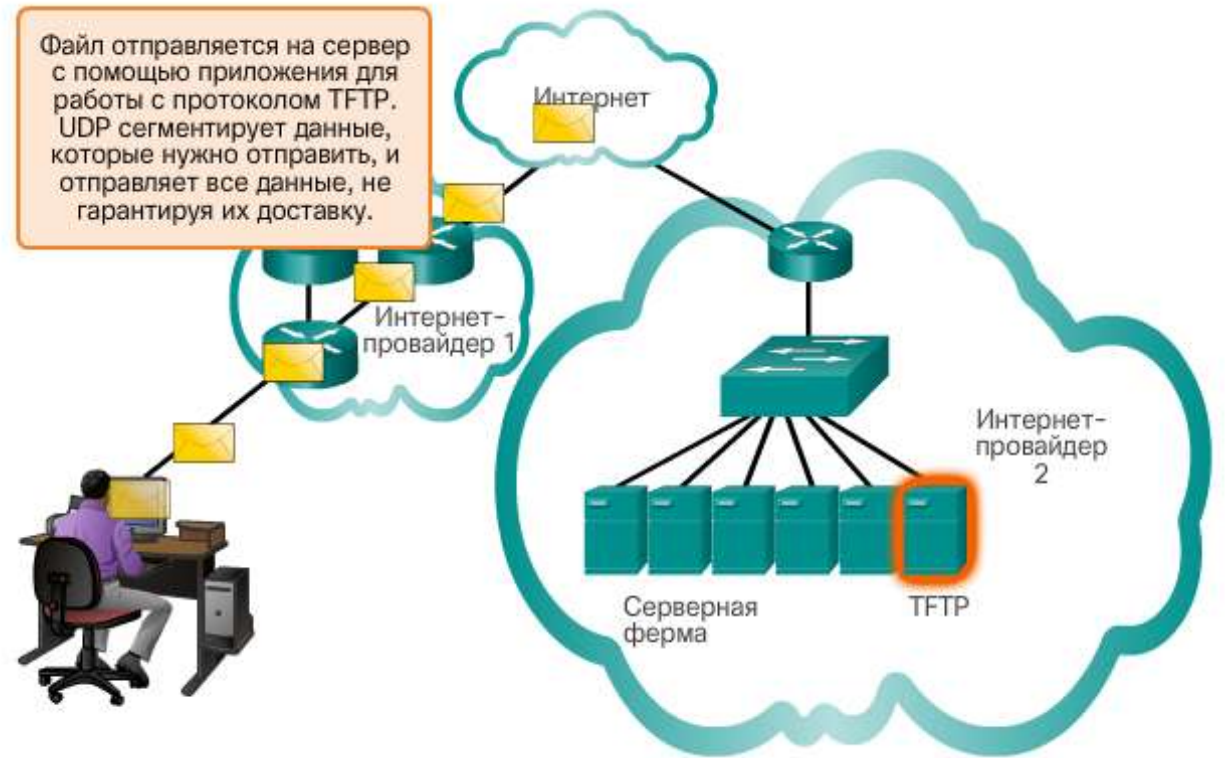


10.1. ПЕРЕДАЧА ДАННЫХ

10.1.6. ПРОТОКОЛ UDP

Некоторые приложения не предъявляют особых требований к надежности. Обеспечение надежности связано с дополнительными накладными расходами и возможными задержками в передаче.

Дополнительные накладные расходы для обеспечения надежности некоторых приложений могут снизить полезность самого приложения и даже отрицательно сказаться на его производительности.

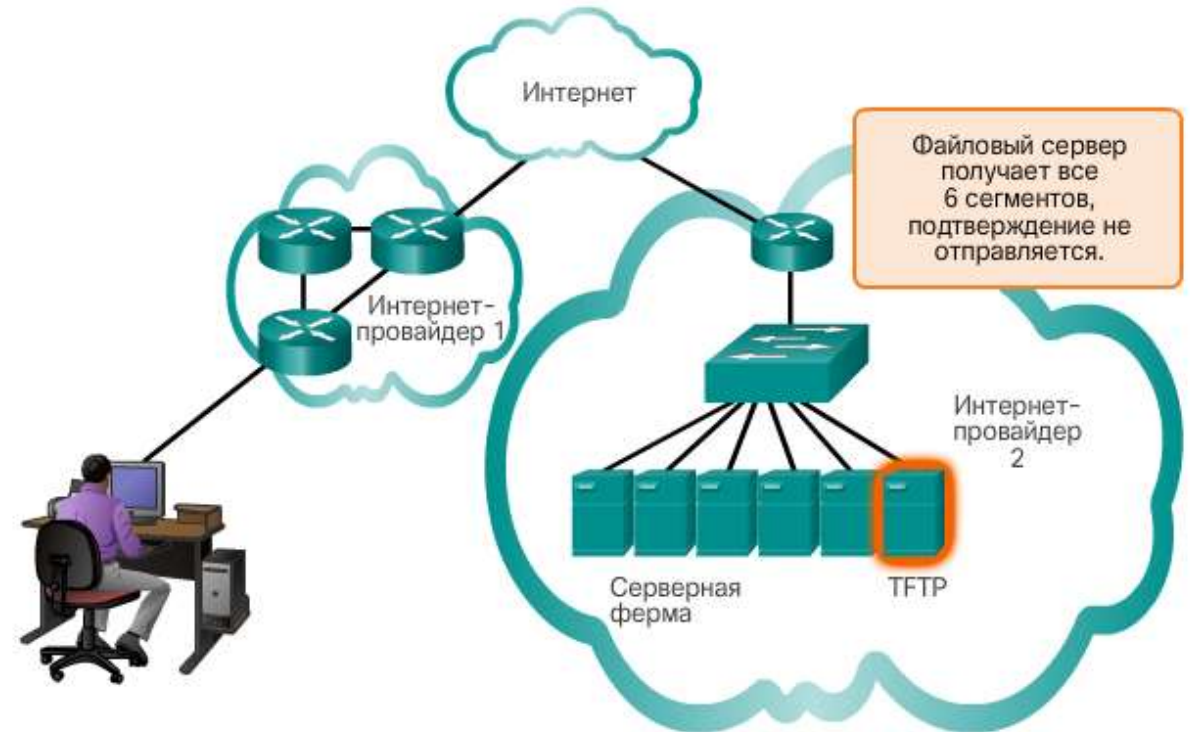


10.1. ПЕРЕДАЧА ДАННЫХ

10.1.6. ПРОТОКОЛ UDP

При отсутствии требований к надежности наиболее подходящим транспортным протоколом для передачи данных является протокол **UDP**.

Он обеспечивает только основные функции для обмена сегментами данных между приложениями. При этом данный протокол создает совсем небольшие накладные расходы и практически не обеспечивает проверку данных.



10.1. ПЕРЕДАЧА ДАННЫХ

10.1.7. СООТВЕТСТВУЮЩИЙ ПРОТОКОЛ ТРАНСПОРТНОГО УРОВНЯ ДЛЯ ПРИЛОЖЕНИЯ

Протокол ТСР лучше всего использовать для следующих приложений:

1. Приложений, которым необходимо, чтобы сегменты передаваемых данных поступали в строго определенной последовательности, в которой они могут быть успешно обработаны
2. Приложениям, которым требуется, чтобы данные были полностью получены прежде, чем их можно будет использовать
3. К приложениям, требующим использования протокола ТСР, относятся: базы данных, веб-браузеры, клиенты электронной почты



Необходимые свойства протоколов:

- Высокая скорость
- Низкие накладные расходы
- Не требуются подтверждения
- Нет повторной отправки потерянных данных
- Доставка данных в том порядке, в каком получится



Необходимые свойства протоколов:

- Надежность
- Подтверждение данных
- Повторная отправка потерянных данных
- Доставка данных в порядке их отправки

10.1. ПЕРЕДАЧА ДАННЫХ

10.1.7. СООТВЕТСТВУЮЩИЙ ПРОТОКОЛ ТРАНСПОРТНОГО УРОВНЯ ДЛЯ ПРИЛОЖЕНИЯ

Протокол UDP лучше всего подходит для тех приложений, в которых потеря некоторых данных во время передачи может быть некритичной, но задержки при передаче являются неприемлемыми

К приложениям, использующим UDP, относятся:

1. Потокковая передача аудио в реальном времени
2. Потокковая передача видео в реальном времени
3. Передача голоса по IP-протоколу (VoIP)



Необходимые свойства протоколов:

- Высокая скорость
- Низкие накладные расходы
- Не требуются подтверждения
- Нет повторной отправки потерянных данных
- Доставка данных в том порядке, в каком получится



Необходимые свойства протоколов:

- Надежность
- Подтверждение данных
- Повторная отправка потерянных данных
- Доставка данных в порядке их отправки

10.2. ОБЗОР ПРОТОКОЛА ТСП

10.2.1. ФУНКЦИИ ПРОТОКОЛА ТСП

Помимо поддержки таких базовых функций, как сегментация данных и повторная компоновка, протокол ТСП также обеспечивает следующие возможности:

1. Установление сеанса связи
2. Надежность доставки
3. Доставка в одинаковом порядке
4. Управление потоком передачи данных

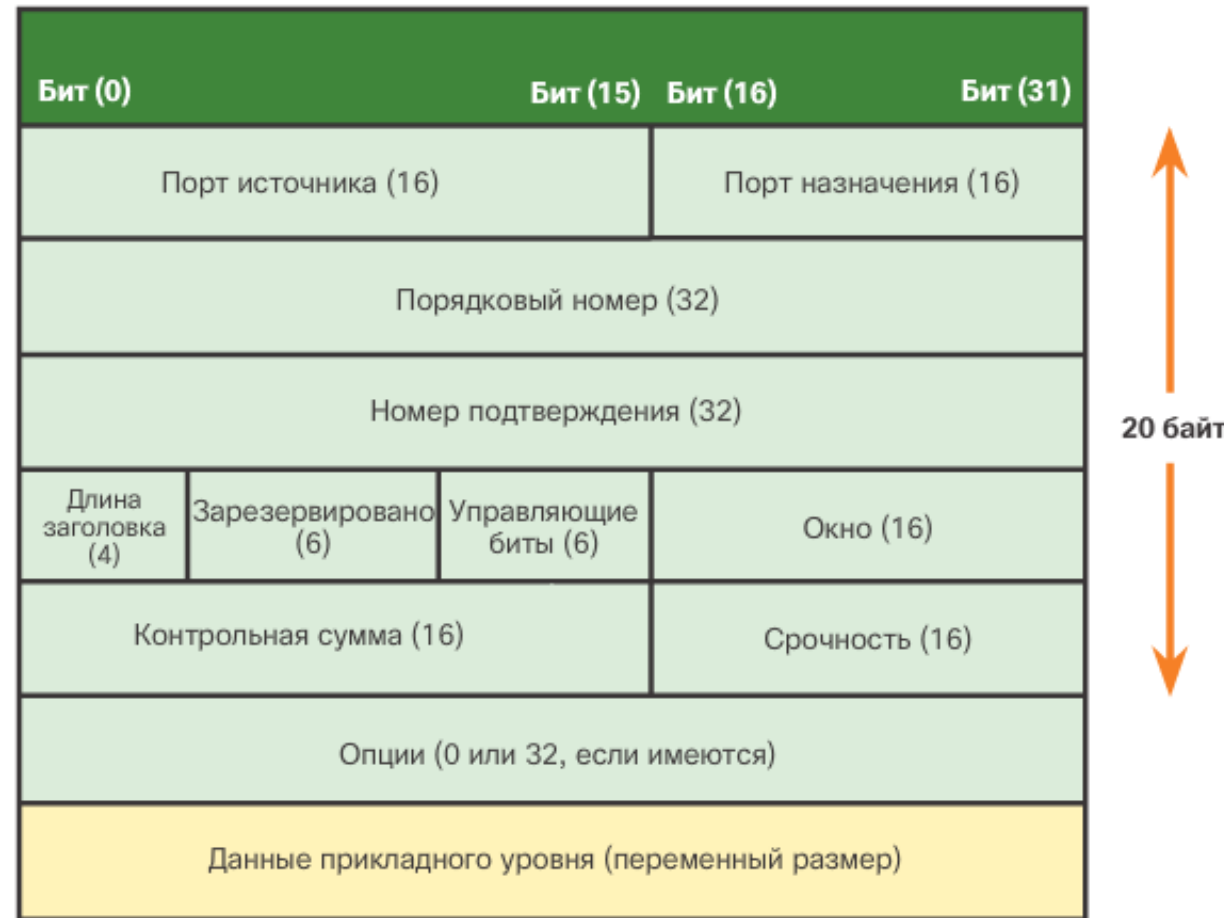


10.2. ОБЗОР ПРОТОКОЛА ТСП

10.2.2. ЗАГОЛОВОК ПРОТОКОЛА ТСП

Протокол ТСП обеспечивает контроль состояния. Для отслеживания состояния сеанса связи протокол ТСП фиксирует, какую информацию он отправил, и какая информация была подтверждена.

Как показано на рисунке, каждый сегмент ТСП содержит 20 дополнительных байт в заголовке, инкапсулируя данные уровня приложений.



10.2. ОБЗОР ПРОТОКОЛА ТСП

10.2.3. ПОЛЯ ЗАГОЛОВКА ТСП

Порядковый номер (32 бита): используется для повторной компоновки данных.

Номер подтверждения (32 бита): обозначает полученные данные.

Длина заголовка (4 бита): параметр, который также называется смещением данных. Обозначает длину заголовка сегмента ТСП.

Зарезервировано (6 бит): поле, зарезервированное для последующего использования.

Биты управления (6 бит): включает двоичные коды, или флаги, которые указывают назначение и функцию сегмента ТСП.

Размер окна (16 бит): отображает количество сегментов, которые можно принять одновременно.

Контрольная сумма (16 бит): используется для проверки ошибок заголовка и данных сегмента.

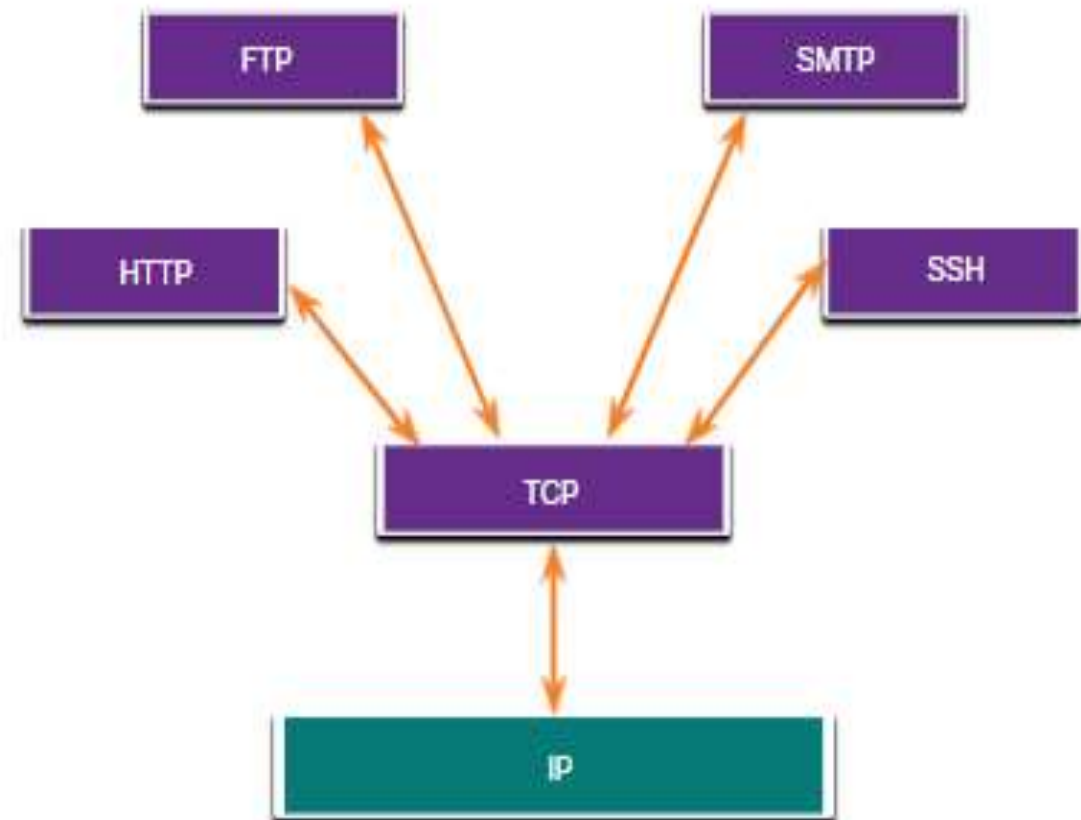
Срочность (16 бит): обозначает, являются ли данные срочными.

В качестве примеров приложений, использующих протокол ТСП, можно привести веб-браузеры, электронную почту и передачи файлов.

10.2. ОБЗОР ПРОТОКОЛА ТСП

10.2.4. ПРИЛОЖЕНИЯ, ИСПОЛЬЗУЮЩИЕ ПРОТОКОЛ ТСП

ТСП сам выполняет все задачи, связанные с разбиением потока данных на сегменты, обеспечением надежности их передачи, управлением потоком и переупорядочением сегментов.



10.3. ОБЗОР UDP

10.3.1. ФУНКЦИИ ПРОТОКОЛА UDP

UDP имеет следующие функции:

1. Данные восстанавливаются в том порядке, в котором получены.
2. Потерянные сегменты повторно не отправляются.
3. Без установления сеанса связи.
4. Без уведомления отправителя о доступности ресурса.



10.3. ОБЗОР UDP

10.3.2. ЗАГОЛОВОК ПРОТОКОЛА UDP

UDP – протокол без отслеживания состояния. Ни отправитель, ни получатель не обязаны отслеживать состояние сеанса связи.

Надежность должна обеспечиваться приложением.

Приложения видео в реальном времени и голосовые приложения должны быстро передавать данные. Они могут допускать потерю некоторых данных и поэтому прекрасно подходят для использования протокола UDP.

Фрагменты данных в протоколе UDP называются **датаграммами**.

Эти датаграммы отправляются без гарантии доставки протоколом транспортного уровня.

Протокол UDP обеспечивает низкие накладные расходы (всего 8 байт).



10.3. ОБЗОР UDP

10.3.2. ЗАГОЛОВОК ПРОТОКОЛА UDP

Таблица идентифицирует и описывает четыре поля в заголовке UDP.

Поле заголовка UDP	Описание
Порт источника	16-битное поле, используемое для идентификации исходного приложения по номеру порта.
Порт назначения	16-битное поле, используемое для идентификации приложения назначения по номеру порта.
Длина	16-битное поле, указывающее длину заголовка датаграммы UDP.
Контрольная сумма	16-битное поле, используемое для проверки ошибок заголовка и данных датаграммы.

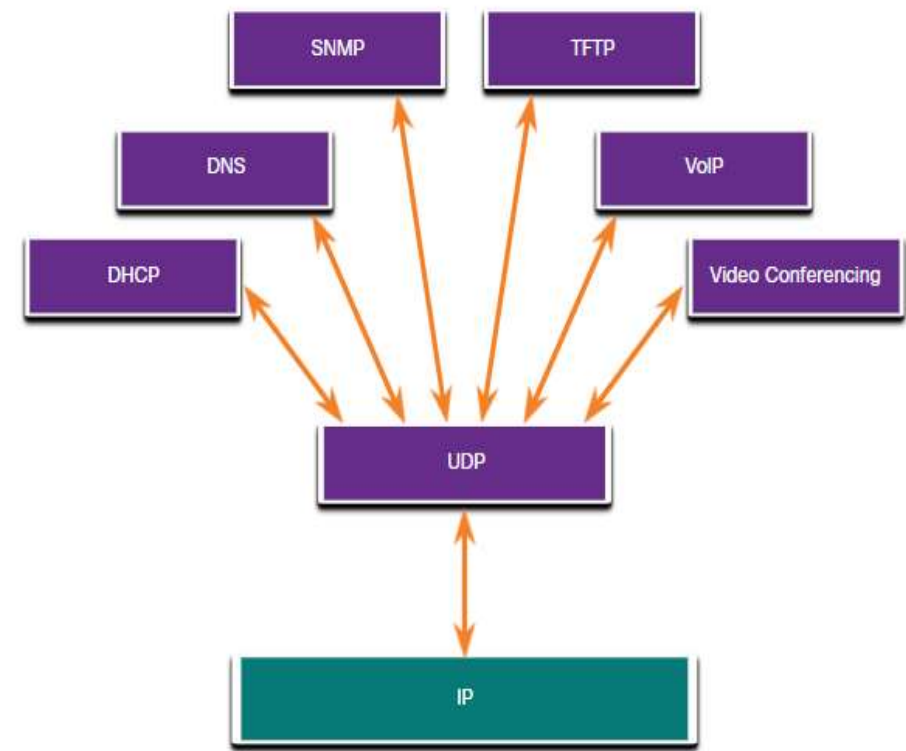
10.3. ОБЗОР UDP

10.3.3. ПРИЛОЖЕНИЯ, ИСПОЛЬЗУЮЩИЕ ПРОТОКОЛ UDP

Мультимедийные приложения и передача видео в режиме реального времени. Такие приложения допускают небольшие потери данных, но не допускают задержки (либо минимальные). Например, VoIP и потоковое видео.

Простые приложения запросов и ответов. Приложения с операциями, где хост отправляет запрос и может получить или не получить ответ. Например, DNS и DHCP.

Приложения, самостоятельно обеспечивающие надежность передачи данных, – ненаправленный обмен данными, при котором управление потоком, обнаружение ошибок, отправка подтверждений и восстановление после сбоев не требуются или выполняются самим приложением. Например, SNMP и TFTP.



10.4. НОМЕРА ПОРТОВ

10.4.1. НЕСКОЛЬКО ОТДЕЛЬНЫХ СЕАНСОВ ПЕРЕДАЧИ ДАННЫХ

Порт источника

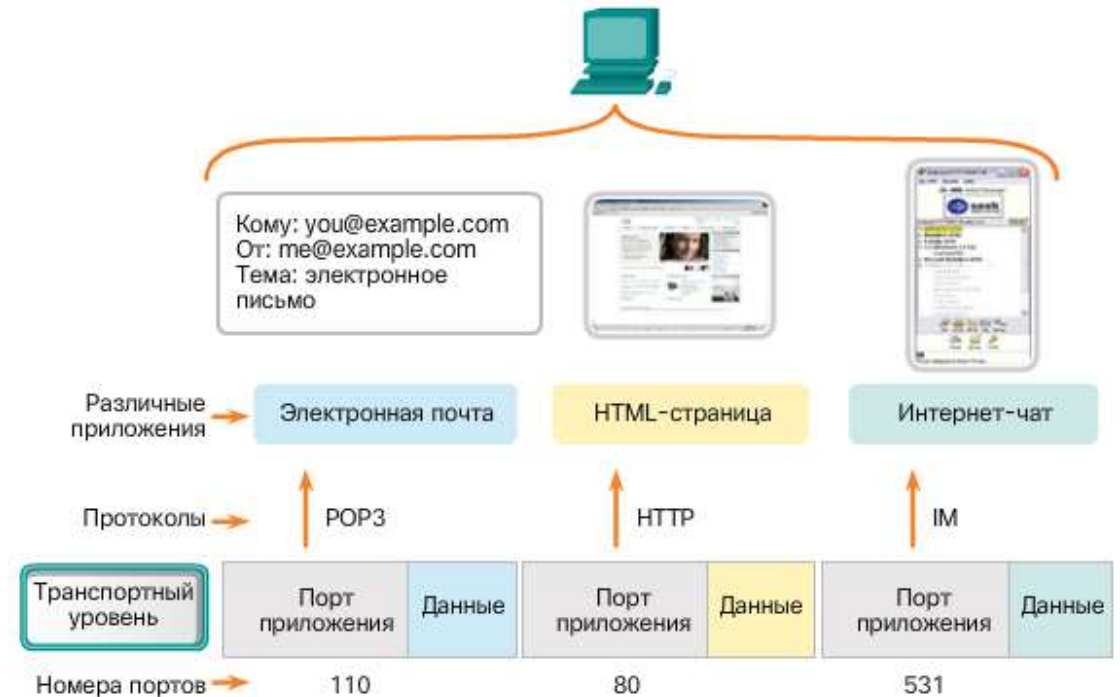
Номер порта источника генерируется динамическим образом устройством-отправителем для идентификации сеанса связи между двумя устройствами.

Клиент HTTP обычно отправляет на веб-сервер несколько запросов сервиса HTTP в одно и то же время. Отдельные сеансы связи по протоколу HTTP отслеживаются по номерам портов источника.

Порт назначения

Используется для идентификации приложения или сервиса, работающих на сервере.

В одно и то же время сервер может предложить больше одного сервиса: одновременно веб-сервис через порт 80 и FTP через порт 21.



10.4. НОМЕРА ПОРТОВ

10.4.2. ПАРЫ СОКЕТОВ

Комбинация IP-адреса источника и номера порта источника или IP-адреса назначения и номера порта назначения называется **сокетом**.

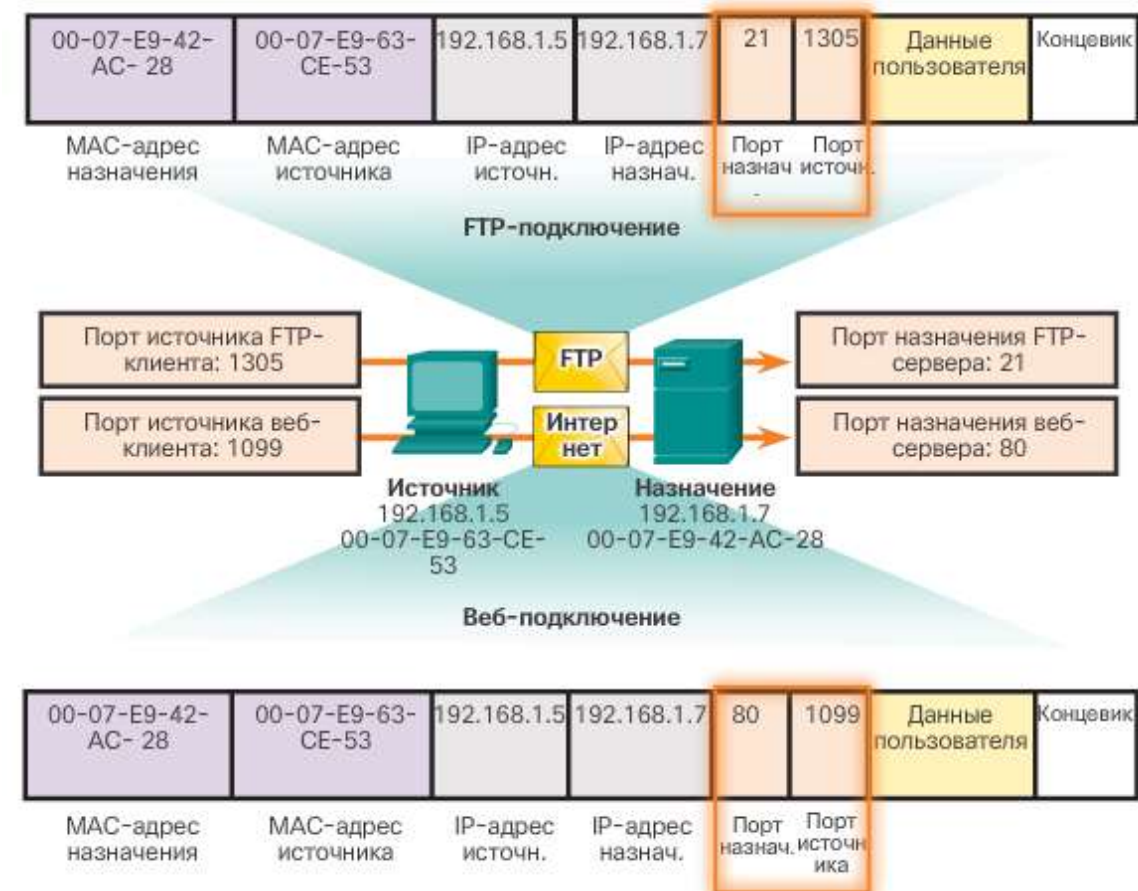
Сокет используется для определения сервера и служб, запрашиваемых клиентом.

Два сокета образуют пару сокетов: (192.168.1.5:1099, 192.168.1.7:80)

Сокеты позволяют различать несколько процессов, выполняющихся на клиенте, а также распознавать различные подключения к процессу сервера.

Номер порта играет роль обратного адреса для запрашивающего приложения.

Отслеживанием активных сокетов занимается транспортный уровень.



10.4. НОМЕРА ПОРТОВ

10.4.3. ГРУППЫ НОМЕРОВ ПОРТОВ

Разработка различных стандартов адресации, включая нумерацию портов, выполняется Администрацией адресного пространства Интернет (Internet Assigned Numbers Authority, **IANA**).

Номера портов



Диапазон номеров портов	Группа портов
От 0 до 1023	Общеизвестные порты
От 1024 до 49151	Зарегистрированные порты
От 49152 до 65535	Частные и/или динамические порты

Общеизвестные номера портов



Номер порта	Протокол	Приложение	Сокращение
20	TCP	File Transfer Protocol (данные)	FTP
21	TCP	File Transfer Protocol (управление)	FTP
22	TCP	Secure Shell	SSH
23	TCP	Telnet	—
25	TCP	Simple Mail Transfer Protocol	SMTP
53	UDP, TCP	Domain Name Service	DNS
67	UDP	Dynamic Host Configuration Protocol (сервер)	DHCP
68	UDP	Dynamic Host Configuration Protocol (клиент)	DHCP
69	UDP	Trivial File Transfer Protocol	TFTP
80	TCP	Hypertext Transfer Protocol	HTTP
110	TCP	Post Office Protocol (версия 3)	POP3
143	TCP	Internet Message Access Protocol	IMAP
161	UDP	Simple Network Management Protocol	SNMP
443	TCP	Hypertext Transfer Protocol Secure	HTTPS

10.4. НОМЕРА ПОРТОВ

10.4.4. КОМАНДА NETSTAT

Неопознанные TCP-соединения могут представлять значительную угрозу безопасности.

Проверить состояние этих подключений помогает важное программное средство — команда **netstat**.

Команда **netstat** позволяет получить список используемых протоколов, локальных адресов и номеров портов, адрес и номер порта на удаленном узле, а также сообщает состояние соединений.

По умолчанию команда **netstat** пытается разрешить IP-адреса в имена доменов, а номера портов — в названия общеизвестных приложений.

Чтобы отобразить IP-адреса и номера портов в числовой форме, можно использовать параметр **-n**.

```
C:\> netstat

Active Connections

Proto  Local Address  Foreign Address  State
TCP    kenpc:3126    192.168.0.2:netbios-ssn  ESTABLISHED
TCP    kenpc:3158    207.138.126.152:http    ESTABLISHED
TCP    kenpc:3159    207.138.126.169:http    ESTABLISHED
TCP    kenpc:3160    207.138.126.169:http    ESTABLISHED
TCP    kenpc:3161    sc.msn.com:http    ESTABLISHED
TCP    kenpc:3166    www.cisco.com:http    ESTABLISHED

C:\>
```

10.5. ОБМЕН ДАННЫМИ ПО ТСП

10.5.1. ПРОЦЕСС ТСП-СЕРВЕРА

Каждый процесс приложения, работающий на сервере, использует **номер порта**.

Не допускается использование двумя различными службами на одном и том же сервере одного и того же порта с одинаковым протоколом транспортного уровня.

Активное серверное приложение, назначенное определенному порту, считается **открытым**.

Любой входящий запрос клиента, адресованный открытому порту, принимается и обрабатывается серверным приложением, связанным с этим портом.

На сервере может быть одновременно открыто сразу несколько портов, по одному для каждого активного приложения сервера.

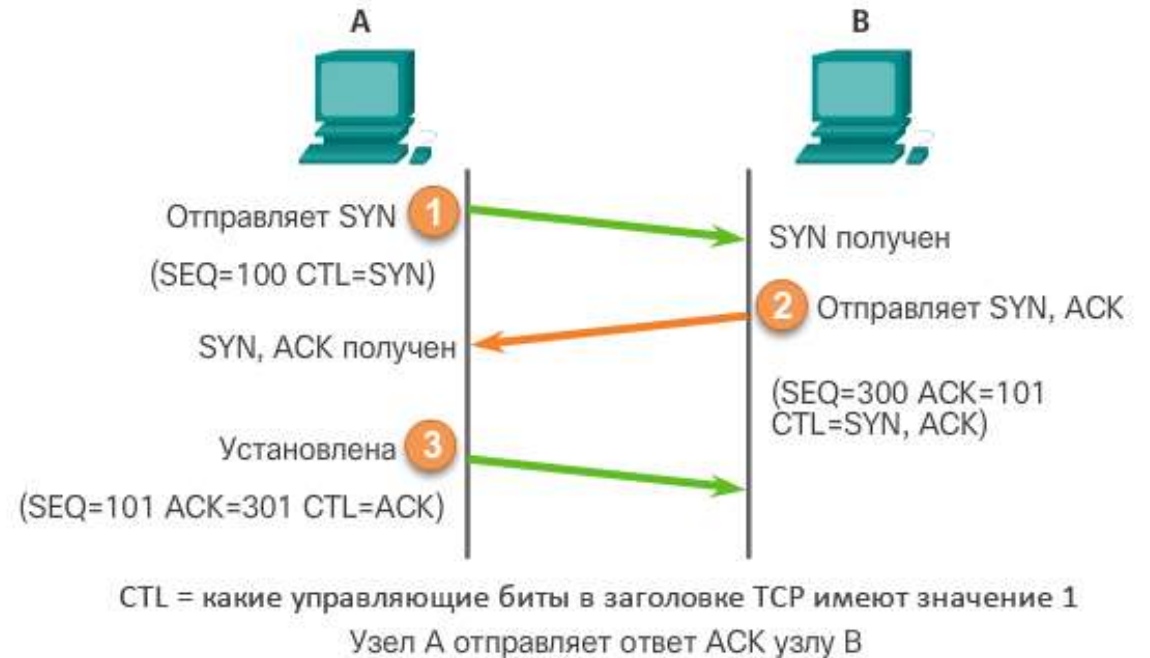


10.5. ОБМЕН ДАННЫМИ ПО ТСР

10.5.2. УСТАНОВЛЕНИЕ ТСР-СОЕДИНЕНИЯ

Установление соединения по протоколу ТСР осуществляется в три этапа:

1. Иницирующий клиент запрашивает сеанс связи типа «клиент-сервер» с сервером
2. Сервер подтверждает сеанс обмена данными «клиент-сервер» и запрашивает сеанс обмена данными «сервер-клиент»
3. Иницирующий клиент подтверждает сеанс обмена данными «сервер-клиент»

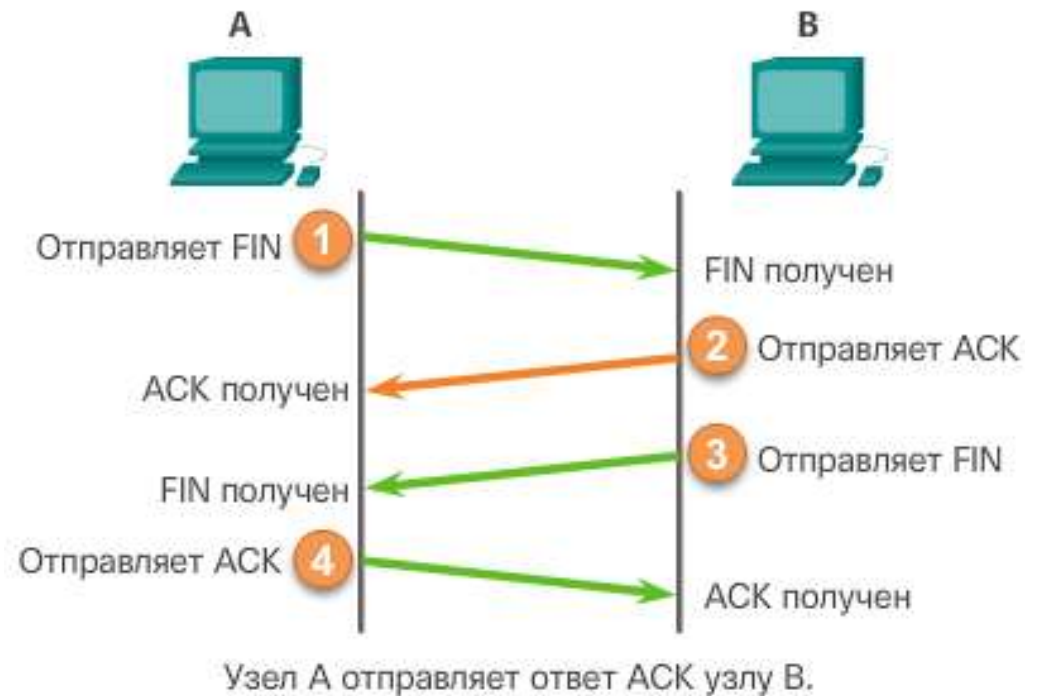


10.5. ОБМЕН ДАННЫМИ ПО ТСП

10.5.3. ПРЕКРАЩЕНИЕ ТСП-СОЕДИНЕНИЯ

Флаг FIN протокола ТСП используется для завершения подключения ТСП:

1. Когда у клиента больше нет данных для отправки в потоке, он отправляет сегмент с установленным флагом FIN
2. Сервер отправляет подтверждение ACK, чтобы подтвердить получение FIN для завершения сеанса связи «клиент-сервер»
3. Сервер отправляет клиенту сегмент FIN, чтобы завершить сеанс связи «сервер-клиент»
4. Клиент отправляет в ответ сегмент ACK для подтверждения получения сегмента FIN от сервера
5. После подтверждения всех сегментов сеанс закрывается

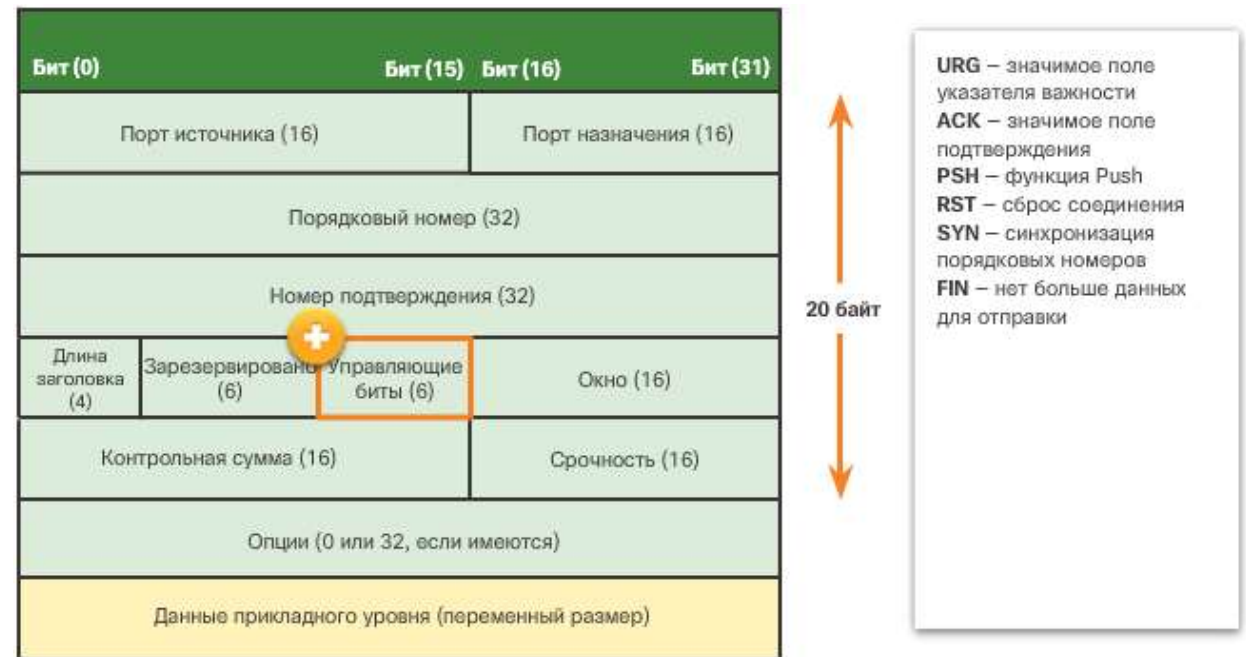


10.5. ОБМЕН ДАННЫМИ ПО ТСП

10.5.4. АНАЛИЗ ТРЕХСТОРОННЕГО КВИТИРОВАНИЯ ТСП

Трехстороннее квитирование:

1. Сначала устанавливается, присутствует ли устройство назначения в сети
2. Затем проверяется, имеется ли на устройстве назначения активный сервис и принимает ли он запросы на номер порта назначения, который инициирующий клиент планирует использовать
3. Далее устройству назначения сообщается, что клиент источника планирует установить сеанс связи на этом номере порта



10.5. ОБМЕН ДАННЫМИ ПО ТСР

10.5.4. АНАЛИЗ ТРЕХСТОРОННЕГО КВИТИРОВАНИЯ ТСР

Шесть контрольных битовых флагов выглядят следующим образом:

URG – флаг «указатель важности».

ACK – флаг подтверждения, используемый при установке соединения и завершении сеанса.

PSH – служит для минимизации задержки при передаче данных от приложения к приложению, заставляя стек протокола немедленно отправлять и доставлять данные, минуя алгоритмы буферизации. Это механизм, который позволяет потоковому протоколу ТСР эффективно работать с приложениями, требующими интерактивности и быстрого отклика.

RST – флаг RST используется для сброса соединения при возникновении ошибки или в случае превышения времени ожидания.

SYN – синхронизировать порядковые номера, используемые при установке соединения.

FIN – больше нет данных от отправителя и используется при завершении сеанса.

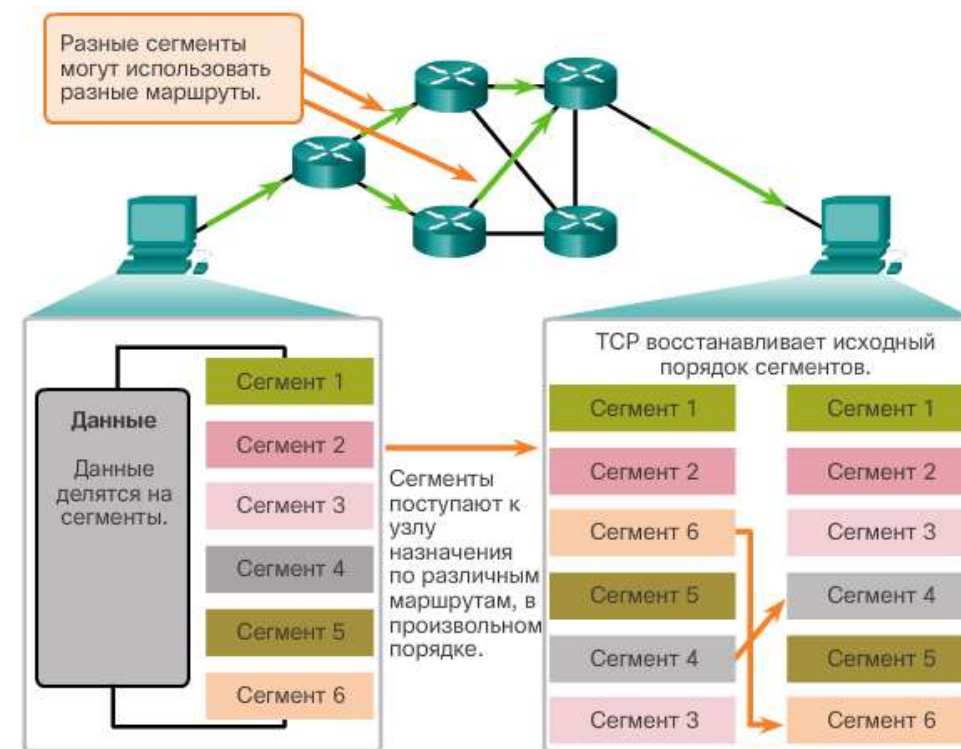
10.6. НАДЕЖНОСТЬ И УПРАВЛЕНИЕ ПОТОКОМ

10.6.1. УПОРЯДОЧЕННАЯ ДОСТАВКА

Сегменты TCP используют **порядковые номера**, чтобы однозначно идентифицировать и подтвердить каждый сегмент, отследить порядок сегментов и указать порядок повторной сборки и переупорядочивания полученных сегментов

Во время настройки сеанса связи задается начальный порядковый номер сеанса (ISN).

Затем ISN увеличивается на количество переданных байтов.



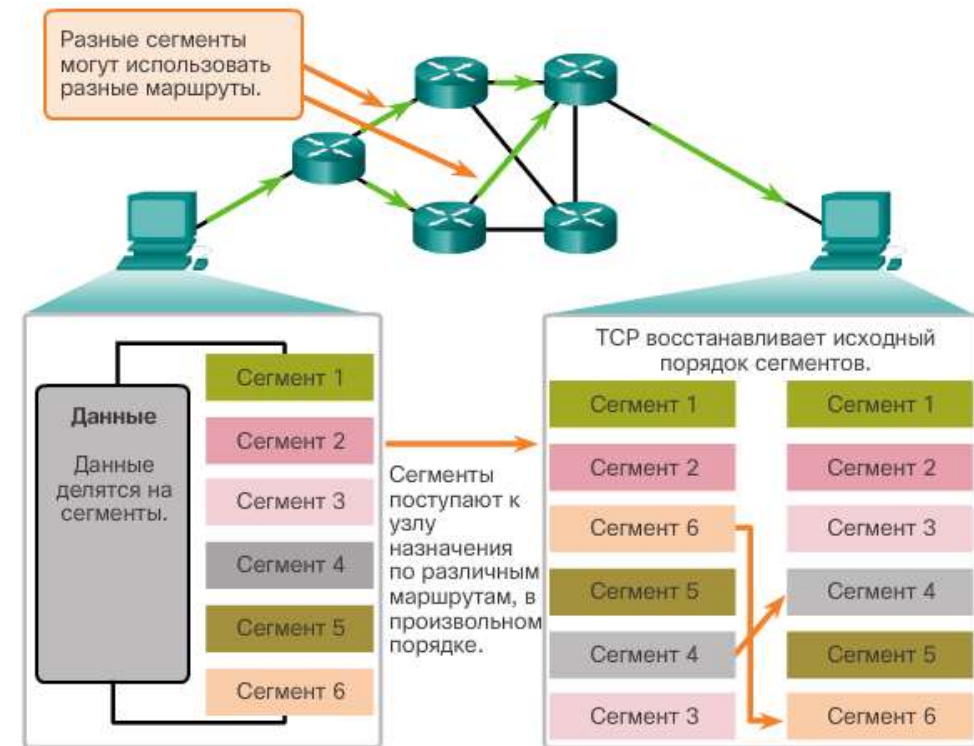
10.6. НАДЕЖНОСТЬ И УПРАВЛЕНИЕ ПОТОКОМ

10.6.1. УПОРЯДОЧЕННАЯ ДОСТАВКА

В ходе получения данных по протоколу TCP данные сегмента помещаются в **буфер** и хранятся там до тех пор, пока все данные не будут получены и повторно собраны.

Сегменты, полученные не в нужном порядке, остаются в буфере для последующей обработки.

Данные передаются на уровень приложений только тогда, когда они будут полностью получены и повторно собраны.

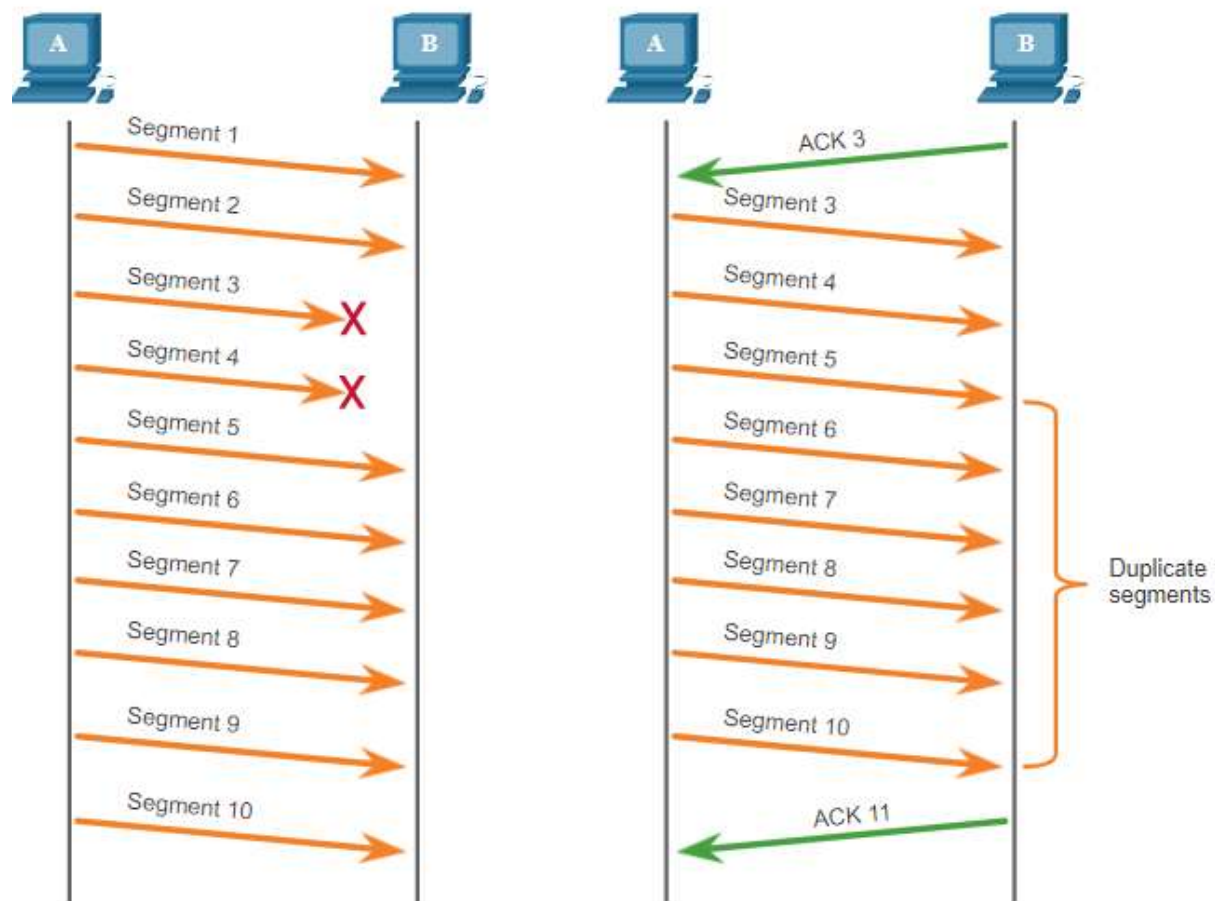


10.6. НАДЕЖНОСТЬ И УПРАВЛЕНИЕ ПОТОКОМ

10.6.2. ПОТЕРЯ ДАННЫХ И ПОВТОРНАЯ ПЕРЕДАЧА

Независимо от того, насколько хорошо разработана сеть, иногда происходит потеря данных.

Протокол TCP обеспечивает возможности для управления потерянными сегментами. Среди них – **механизм повторной передачи** сегментов с данными, получение которых не было подтверждено.

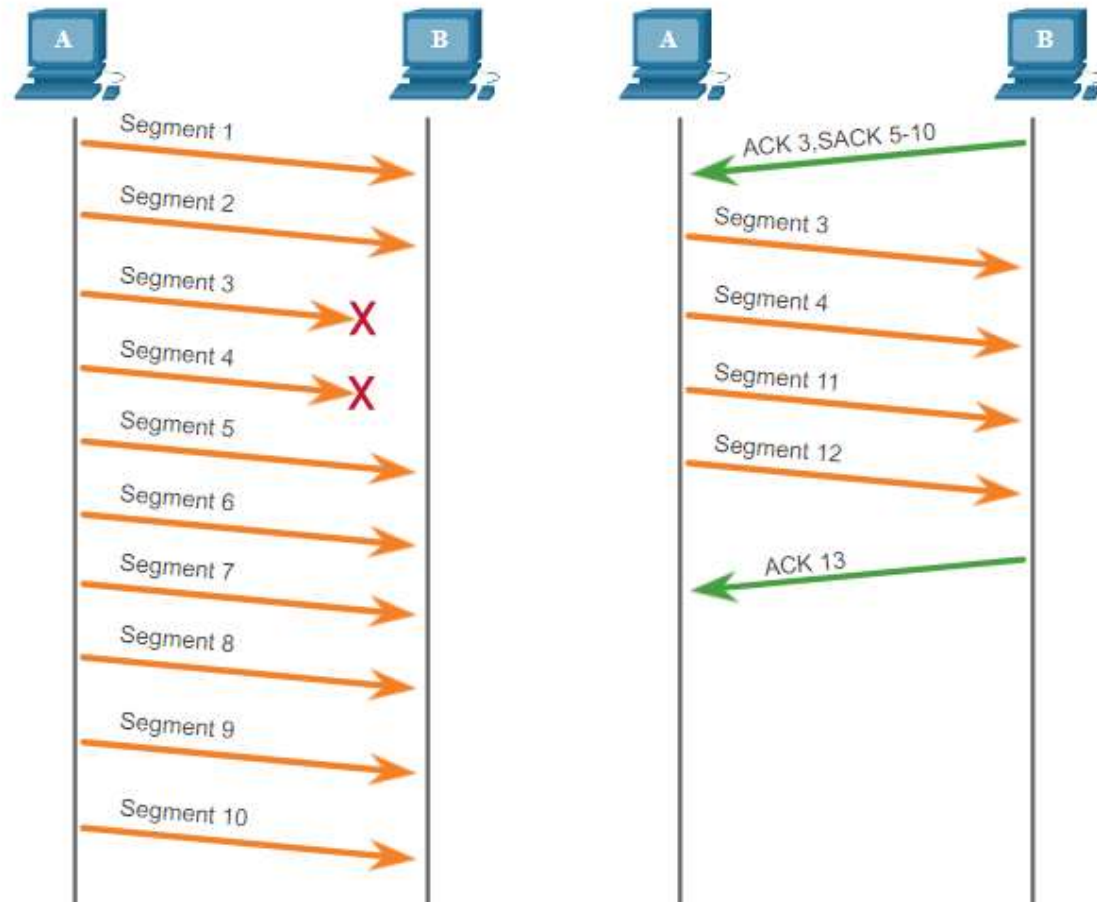


10.6. НАДЕЖНОСТЬ И УПРАВЛЕНИЕ ПОТОКОМ

10.6.2. ПОТЕРЯ ДАННЫХ И ПОВТОРНАЯ ПЕРЕДАЧА

В настоящее время серверные операционные системы обычно используют опциональную функцию TCP, называемую **выборочным подтверждением (SACK)**, согласованную во время трехстороннего рукопожатия.

Если оба узла поддерживают SACK, приемник может явно определить, какие сегменты (байты) были получены, включая любые сегменты прерывания.



10.6. НАДЕЖНОСТЬ И УПРАВЛЕНИЕ ПОТОКОМ

10.6.3. РАЗМЕР ОКНА И ПОДТВЕРЖДЕНИЯ

Протокол ТСР обеспечивает механизмы для управления потоком данных

Управление потоком обеспечивает возможность надежного получения и обработки данных оконечными устройствами ТСР.

Протокол ТСР осуществляет управление потоком путем регулировки скорости потока данных между узлами источника и назначения в течение отдельно взятого сеанса.

Функция управления потоком ТСР основывается на использовании 16-битового поля заголовка ТСР, называемого размером окна. **Размер окна** – это количество байтов, которое устройство назначения способно принять и обработать за один раз во время сеанса ТСР.

Узел источника и узел назначения протокола ТСР согласовывают начальный размер окна при установлении сеанса ТСР.

Оконечные устройства ТСР могут настроить размер окна во время сеанса, если это необходимо.

10.6. НАДЕЖНОСТЬ И УПРАВЛЕНИЕ ПОТОКОМ

10.6.3. РАЗМЕР ОКНА И ПОДТВЕРЖДЕНИЯ

Максимальный размер сегмента (MSS) – это максимальный объем данных, который может получить конечное устройство.

Обычный MSS составляет 1 460 байт при использовании IPv4.

Узел определяет значение своего поля MSS путем вычитания размера заголовков IP и TCP из размера MTU для Ethernet.

1500 минус 40 (20 байт для заголовка IPv4 и 20 байт для заголовка TCP) оставляет 1460 байт.



С помощью **размера окна** определяется количество байтов, отправленных до того, как ожидается подтверждение.

Номер **подтверждения** – это номер следующего ожидаемого байта.

10.6. НАДЕЖНОСТЬ И УПРАВЛЕНИЕ ПОТОКОМ

10.6.4. ПРЕДОТВРАЩЕНИЕ ПЕРЕГРУЗОК

Перегрузка сети обычно приводит к сбросу пакетов.

Недоставленные сегменты ТСП инициируют повторную передачу. Повторная передача сегмента ТСП может привести к еще более сильной перегрузке сети.

Определив скорость передачи данных, при которой сегменты ТСП отправляются, но не подтверждаются, узел источника может примерно определить уровень загрузки сети.

При обнаружении перегрузки сети узел источника может уменьшить количество отправляемых им байтов до получения подтверждения.

Узел источника сокращает именно **количество неподтвержденных байтов**, которые он отправляет, а не размер окна, определенный узлом назначения.

Узел назначения, как правило, не знает о загрузке сети и не видит необходимости в изменении размера окна.

10.6. НАДЕЖНОСТЬ И УПРАВЛЕНИЕ ПОТОКОМ

10.6.4. ПРЕДОТВРАЩЕНИЕ ПЕРЕГРУЗОК



Подтверждения нумеруются по следующему полученному байту, а не по номеру сегмента. Номера сегментов упрощены в целях наглядности.

10.7. ОБМЕН ДАННЫМИ ПО UDP

10.7.1. UDP: НИЗКАЯ НАГРУЗКА ИЛИ НАДЕЖНОСТЬ

UDP – это простой протокол, он обеспечивает базовые функции транспортного уровня.

Протокол UDP связан с гораздо меньшими накладными расходами, чем протокол TCP, и не является протоколом с установлением подключения. В нем не предусмотрены сложные механизмы повторной отправки, упорядочивания и управления потоком данных.

Приложения, работающие по протоколу UDP, могут по-прежнему использоваться в качестве надежных, но их надежность должна быть реализована на уровне приложений.

Однако это не уменьшает значимость протокола UDP. Он специально спроектирован так, чтобы быть быстрее и проще протокола TCP за счет снижения надежности.



UDP не устанавливает соединение перед отправкой данных.

10.7. ОБМЕН ДАННЫМИ ПО UDP

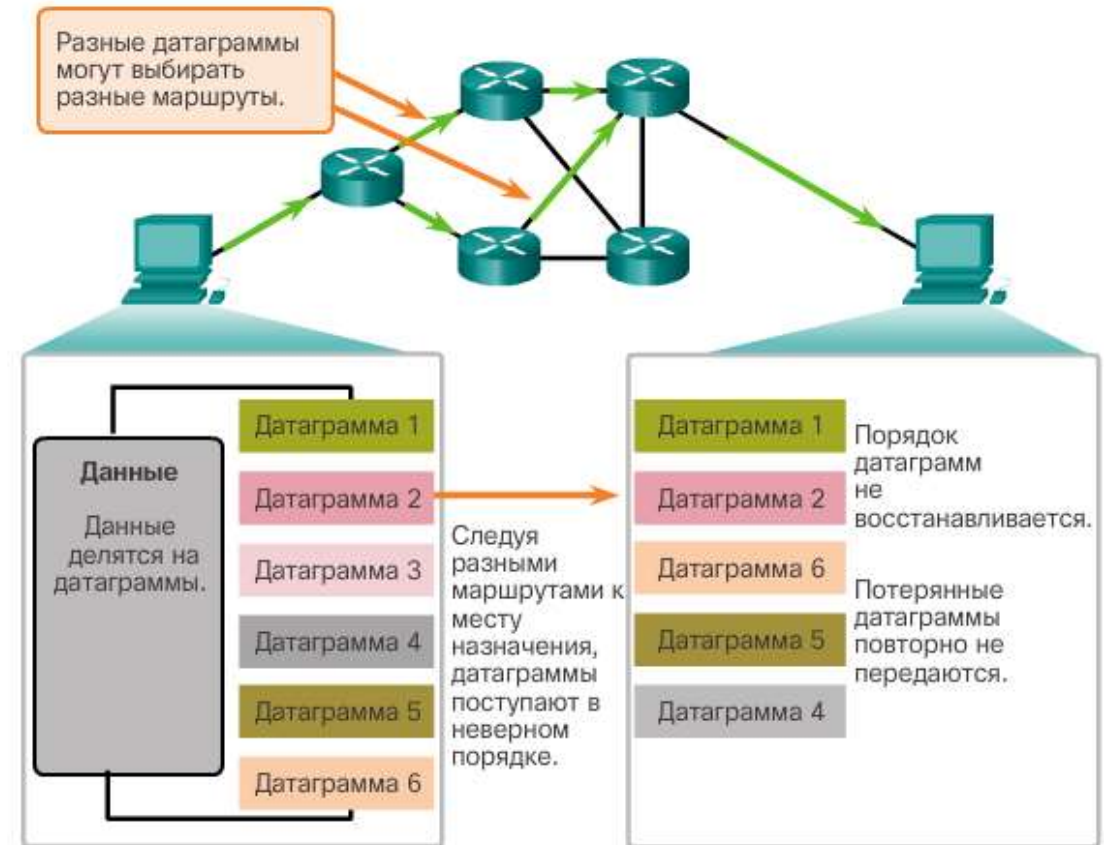
10.7.2. ПОВТОРНАЯ ДЕФРАГМЕНТАЦИЯ ДАТАГРАММЫ UDP

Протокол UDP не отслеживает порядковые номера, как это делает протокол TCP.

Протокол UDP не может повторно скомпоновать датаграммы в том порядке, который использовался при их передаче

Протокол UDP просто повторно собирает данные в том порядке, в котором они были приняты.

Правильную последовательность должно определять приложение, если это необходимо.



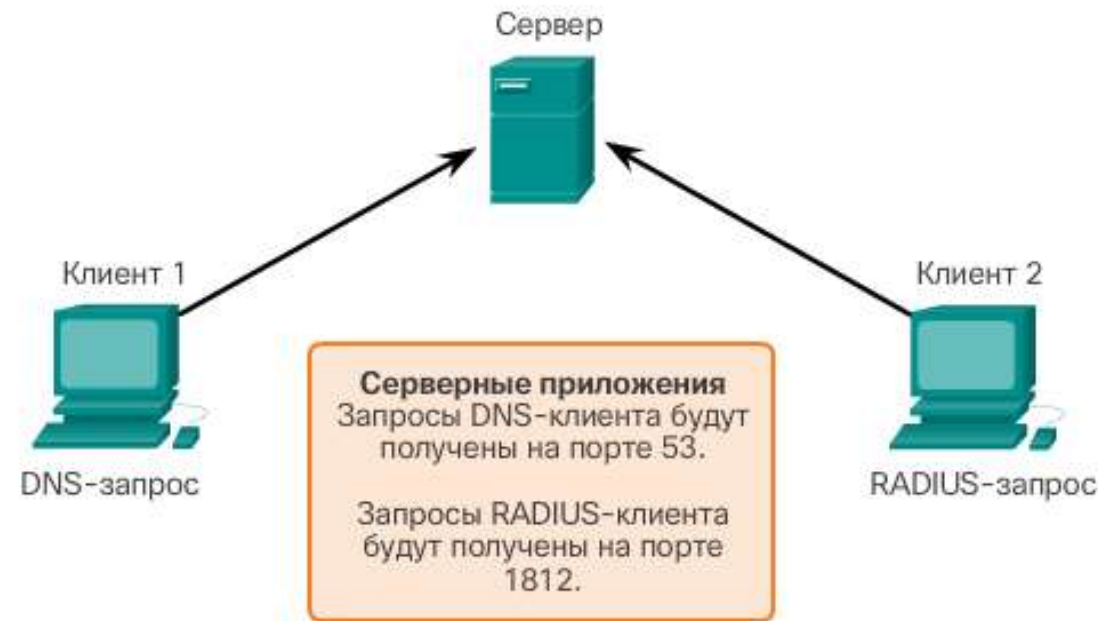
10.7. ОБМЕН ДАННЫМИ ПО UDP

10.7.3. ПРОЦЕССЫ И ЗАПРОСЫ UDP-СЕРВЕРА

Серверным приложениям на базе UDP назначаются широко известные или зарегистрированные номера портов.

Приложения и сервисы протокола UDP, работающие на сервере, принимают клиентские запросы UDP.

Запросы, полученные через определенный порт, направляются к нужному приложению на основе **номеров портов**.



10.7. ОБМЕН ДАННЫМИ ПО UDP

10.7.3. ПРОЦЕССЫ И ЗАПРОСЫ UDP-СЕРВЕРА

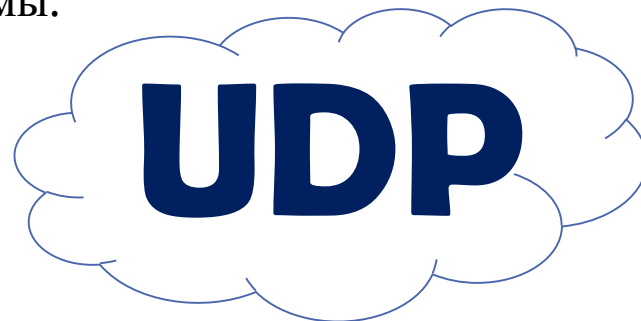
Передача данных «клиент-сервер» по протоколу UDP также инициируется клиентским приложением.

Процесс UDP-клиента динамическим образом выбирает номер порта и использует его в качестве порта источника.

Как правило, порт назначения – это общеизвестный или зарегистрированный номер порта, присвоенный процессу сервера.

В заголовке всех датаграмм, используемых в процессе пересылки, используется одна и та же пара портов узла источника и узла назначения.

Данные, возвращаемые клиенту от сервера, используют номера портов узла источника и узла назначения в заголовке датаграммы.



ВОПРОСЫ ДЛЯ ПРОВЕРКИ



1. За что отвечает транспортный уровень?
2. С какими приложениями работает протокол TCP?
3. С какими приложениями работает протокол UDP?
4. Назовите функции протокола TCP.
5. Что из себя представляет размер окна в протоколе TCP?
6. Назовите функции протокола UDP.
7. Сколько байт содержат заголовки TCP и UDP?
8. Для чего используются порты?
9. Что такое сокет?
10. Как происходит установка соединения по протоколу TCP?